

Auto-Memory System

Introduced: v2.1.57 (March 2026) | Latest: v2.1.76

What It Is

Claude Code now has a persistent, file-based memory system that survives across conversations. When you share preferences, correct Claude's behavior, or discuss project context, Claude can save that information to memory files. Future sessions automatically load relevant memories, so Claude doesn't start from scratch every time.

Why It Matters

Before auto-memory, every new Claude Code session was a blank slate: - You'd repeat the same preferences ("use Bun not npm") - Re-explain project context ("we're in a code freeze until Thursday") - Re-correct the same mistakes ("don't mock the database in tests")

Now Claude builds up understanding of you and your projects over time.

Quick Start (60 Seconds)

You: remember that I prefer dark theme UIs and I'm a senior backend engineer

Claude: [saves to memory – future sessions will know this]

Or Claude saves memories automatically when it learns something important about you, your preferences, or your project.

Memory Architecture

Storage Location

```
~/ .claude/projects/<project-path>/memory/  
├─ MEMORY.md           # Index file (always loaded, max 200 lines)  
├─ user_role.md         # Memory: user is senior backend engineer  
├─ feedback_testing.md  # Memory: don't mock databases in tests  
├─ project_freeze.md    # Memory: code freeze until March 20  
└─ reference_linear.md   # Memory: bugs tracked in Linear project INGEST
```

The MEMORY.md file is an index — it contains only links to individual memory files with brief descriptions. It's always loaded into context.

Custom Directory

```
{  
  "autoMemoryDirectory": "~/my-custom-memory-dir/"  
}
```

Memory File Format

Each memory file uses YAML frontmatter:

```
---
name: Testing Preferences
description: Integration tests must use real database, not mocks
type: feedback
---
```

Integration tests must hit a real database, not mocks.

****Why:**** Prior incident where mock/prod divergence masked a broken migration.

****How to apply:**** When writing or reviewing test code, always use real DB connections.

Memory Types

Type	What it stores	When saved	How used
user	Role, goals, preferences, knowledge level	When learning about the user	Tailor responses to user's expertise
feedback	Corrections, behavior guidance	When user corrects Claude's approach	Avoid repeating same mistakes
project	Ongoing work, deadlines, initiatives	When learning project context	Better informed suggestions
reference	Pointers to external systems	When learning about external resources	Know where to look for information

User Memory Examples

"I'm a data scientist investigating logging"

→ saves: user is data scientist, focused on observability

"I've been writing Go for ten years but this is my first React project"

→ saves: deep Go expertise, new to React – frame frontend in backend terms

Feedback Memory Examples

"Don't mock the database in tests"

→ saves: integration tests must use real DB (reason: mock/prod divergence incident)

"Stop summarizing at the end of every response"

→ saves: terse responses, no trailing summaries

Project Memory Examples

"We're freezing merges after Thursday for mobile release"

→ saves: merge freeze 2026-03-20 for mobile release cut

"Legal flagged the old auth middleware for compliance"

→ saves: auth rewrite driven by legal/compliance, not tech debt

Reference Memory Examples

"Check Linear project INGEST for pipeline bugs"

→ saves: pipeline bugs tracked in Linear project "INGEST"

"The Grafana board at grafana.internal/d/api-latency is what oncall watches"

→ saves: check grafana.internal/d/api-latency when editing request-path code

What NOT to Save

Memory is for information that **can't be derived** from the codebase:

Don't save	Why	Use instead
Code patterns, architecture	Derivable from reading code	Read the files
Git history, who-changed-what	Authoritative in git	git log, git blame
Debugging solutions	Fix is in the code	Commit messages
Anything in CLAUDE.md	Already loaded every session	CLAUDE.md
Ephemeral task details	Only relevant now	Tasks tool

Memory vs Other Persistence

Mechanism	Scope	Persists across sessions?	Best for
Memory	Cross-session	Yes	User prefs, feedback, project context
Tasks	Current session	No	Breaking work into steps, tracking progress
Plans	Current session	No	Aligning on implementation approach
CLAUDE.md	Always loaded	Yes (manual)	Project rules, conventions

Real-World Workflows

Workflow 1: Onboarding Memory

You: I'm new to this team. I mainly work on the payment service.

We use Bun, Hono for HTTP, and Drizzle for ORM.

Tests go in __tests__ folders, not separate test/ directories.

Claude: [saves user memory + project memory]

Future sessions know your role and project conventions without re-explaining.

Workflow 2: Feedback Loop

You: Don't add try-catch blocks everywhere, our error boundary handles that

Claude: [saves feedback memory]

Next time Claude writes code, it won't wrap everything in try-catch.

Workflow 3: Cross-Session Context

Session 1: "We're migrating from REST to GraphQL, starting with user endpoints"
→ Claude saves project memory

Session 2: "Add the profile endpoint"
→ Claude knows to build it as GraphQL, not REST

Managing Memory

Explicit Save

remember that I prefer functional components over class components

Explicit Forget

forget that I prefer dark theme

Check Memory

what do you remember about me?
what's in your memory about this project?

Memory is Editable

Memory files are plain markdown — you can edit them directly:

```
vim ~/.claude/projects/-Users-me-myproject/memory/user_role.md
```

How It Interacts With Other Features

Feature	Interaction
CLAUDE.md	Memory supplements CLAUDE.md — don't duplicate what's already there
Session Resume	Memory persists independently. Resumed sessions also load memories
Context Compaction	Memory files are re-read after compaction if needed
Skills	Skills can reference memory for user-specific behavior
Agent Teams	Each teammate loads the same memory files
Worktrees	Memory is project-level, shared across worktrees

Gotchas & Tips

1. **200-line MEMORY.md limit** — keep the index concise. Lines after 200 are truncated.
 2. **Relative dates** — Claude converts “Thursday” to absolute dates (e.g., “2026-03-20”) so memories stay meaningful over time.
 3. **Memory is not a database** — it’s for high-signal information. Don’t ask Claude to remember every detail.
 4. **Duplicate prevention** — Claude checks for existing memories before creating new ones. Existing memories get updated rather than duplicated.
 5. **Memory decay** — project memories can become stale. Claude uses timestamps and “Why” context to judge whether a memory is still relevant.
 6. **autoMemoryDirectory setting** — configure a custom directory if you want memories stored elsewhere.
-

Version History

Version	Date	Change
v2.1.57	Mar 2026	Auto-memory system introduced
v2.1.60	Mar 2026	Memory timestamps for freshness reasoning
v2.1.63	Mar 2026	autoMemoryDirectory setting
v2.1.76	Mar 2026	Memory stability improvements, better duplicate detection